



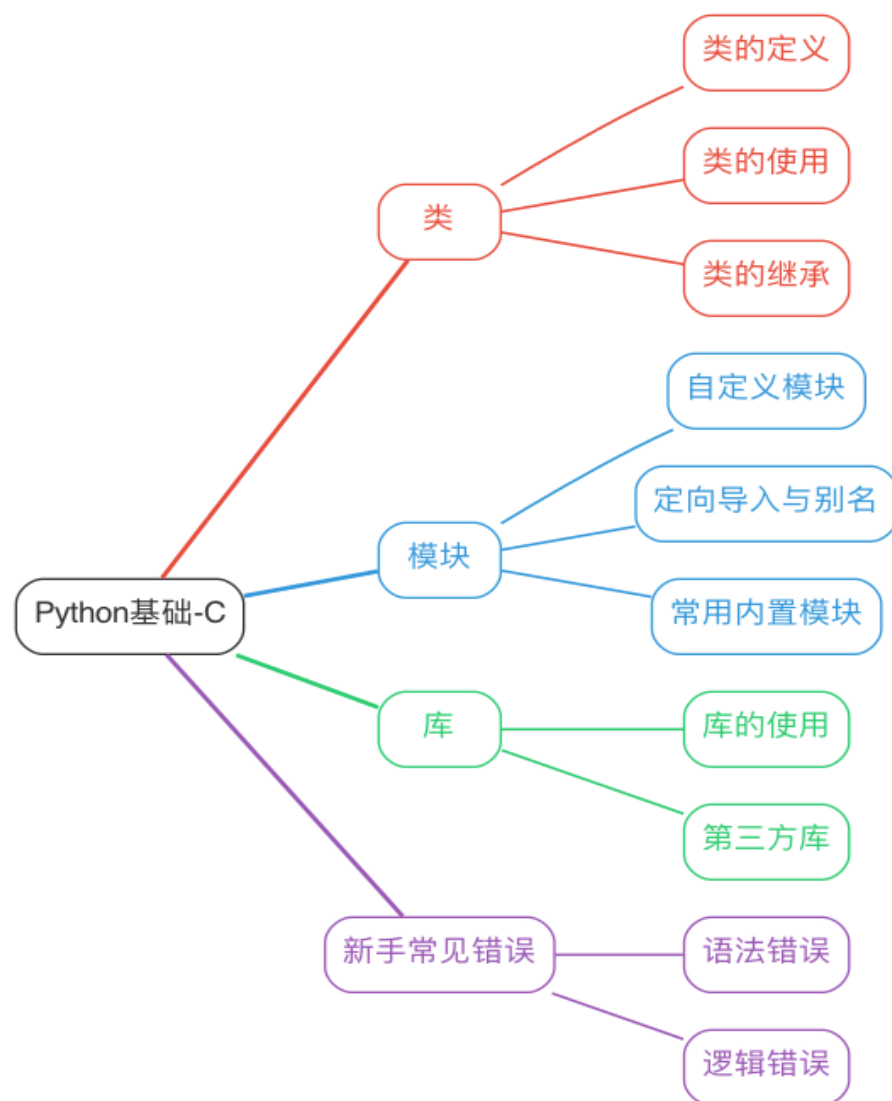
《实用Python编程》

第5讲：Python基础-C

(类、模块、库及新手常见错误)

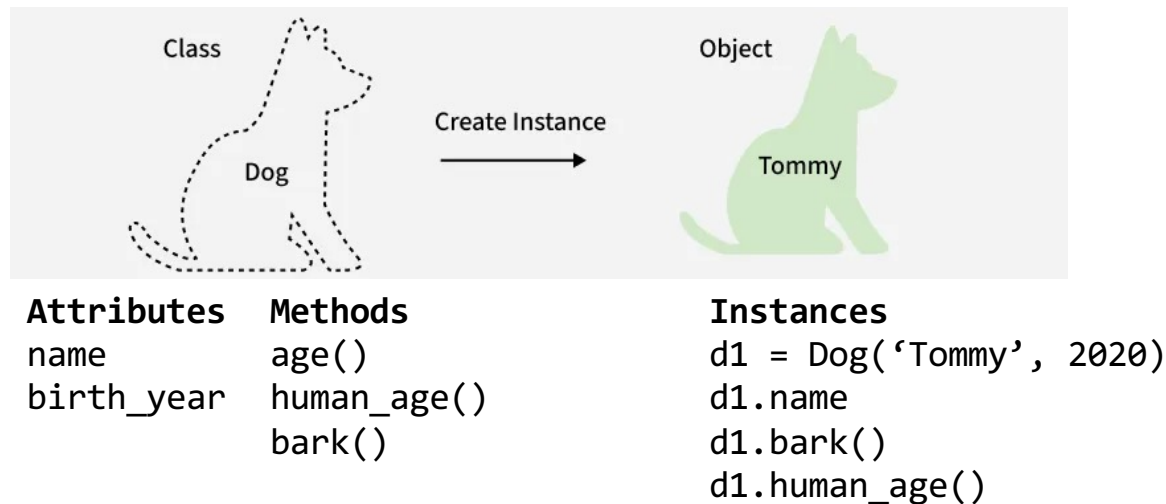
实用Python编程 教研组

提纲



类

- 类(Class)是一种将数据和操作数据的函数封装在一起的数据类型
 - 属性(Attribute): 被类封装的数据
 - 方法(Method): 定义在类内部的函数



面向对象编程(Object-Oriented Programming, OOP)的基础概念与核心工具

类的定义

- 以保留字`class`打头，空格之后跟着的是类名
- 类的命名规范
 - 大驼峰命名法 (UpperCamelCase)，每个单词首字母大写，没有下划线
 - 例如，Person、PageDownloader
- 保留字`self`代表该类的一个实例对象 (instance)
- 类方法分为
 - 实例方法：只能由实例调用
 - 静态方法：实例或类均可调用。以`@staticmethod`装饰

```
1 # 类定义开始
2 class MyClassName: # 类名
3     def __init__(self, 参数1, 参数2, ...): # 构造方法
4         self.属性1 = 参数1
5         self.属性2 = 参数2
6
7     def method1(self, 参数): # 实例方法，第一个参数必须是self
8         函数体
9         print(self.属性1)
10
11     @staticmethod
12     def method2(参数): # 静态方法，以@staticmethod装饰，无self
13         函数体
14 # 类定义结束
```

注意缩进



类的使用

- 通过创建一个关于该类的实例对象来使用

示例代码 2.34 自定义一个新类的基础语法

```
1 # 类定义开始
2 class MyClassName: # 类名
3     def __init__(self, 参数1, 参数2, ...): # 构造方法
4         self.属性1 = 参数1
5         self.属性2 = 参数2
6
7     def method1(self, 参数): # 实例方法, 第一个参数必须是self
8         函数体
9         print(self.属性1)
10
11     @staticmethod
12     def method2(参数): # 静态方法, 以staticmethod装饰, 无self
13         函数体
14 # 类定义结束
15 # 为该类创建对象
16 obj1 = MyClassName(...)
17 obj2 = MyClassName(...)
18 obj1.method1()
19 obj1.method2()
20 MyClassName.method2()
21 MyClassName.method1() # 不允许
```

示例：定义一个Person类

设计思路

- 对于Person而言，出生日期是基本属性
 - 定义一个三元组
`birth`记录该属性
- 年龄随时间增长，需要实时计算
 - 定义一个实例方法
`get_age`
- 判断是否成年并非实例专属
 - 定义一个静态方法
`is_adult`

```
1 from datetime import date # 从 Python内置的datetime 模块中导入date类, 用于计算时间
2
3 class Person:
4     def __init__(self, birth_year, birth_month, birth_day):
5         self.birth = (birth_year, birth_month, birth_day)
6
7     def get_age(self):
8         today = date.today()
9         age = today.year - self.birth[0]
10        # 根据出生月份及日期对age进行必要的修正
11        if (today.month < self.birth[1]) or (today.month == self.birth[1] and today.day < self.birth[2]):
12            age = age - 1
13        return age
14
15    @staticmethod
16    def is_adult(age):
17        return age >= 18
18
19 p = Person(2001, 10, 1) # 创建一个该类的对象
20 print(f'年龄: {p.get_age()}') # 输出: 年龄: 24
21 age = p.get_age()
22 print(p.is_adult(age), Person.is_adult(age)) # 输出: True, True
```

年龄: 24
True True



示例：定义一个Dog类并创建多个实例

- 类方法作为一种类内部的函数，仍遵循自定义函数的基本规则
- 一个方法可以调用另一个方法
 - 年龄换算方法`human_age`调用了`age`方法

```
1 class Dog:
2     def __init__(self, name, birth_year):
3         self.name = name
4         self.birth_year = birth_year
5
6     def bark(self): # 类方法, 无返回值
7         print(f"{self.name} says: Woof!")
8
9     def age(self): # 返回年龄
10        age = date.today().year - self.birth_year
11        return age
12
13    def human_age(self): # 换算成人的年龄返回
14        return self.age() * 7
15
16 # 创建类的实例 (对象)
17 d1 = Dog('Fluppy', 2020)
18 d2 = Dog('Max', 2021)
19 # 访问属性和调用方法
20 print(d1.name, d2.name) # 输出 Fluppy, Max
21 d1.bark() # 输出 Fluppy says: Woof!
22 print(f'age {d2.age()}, human age {d2.human_age()}') # 输出: age 4, human age 28
23
24 Fluppy Max
25 Fluppy says: Woof!
26 age 5, human age 35
```



课堂练习-2-3-1

实现一个微信钱包类Wallet，支持三个日常动作：

- 充值 top_up：将指定金额存入钱包；
- 发红包 send_hongbao：从钱包中扣钱。若钱不够，提示“余额不足”；
- 查余额 check_balance：返回钱包余额

测试用例如下：

```
wallet = Wallet()
wallet.top_up(10) # 一开始存入10元

for val in [-5, 2, -10]: # val>0表示充值, val<0表示发红包
    # code here
print(f'零钱: {wallet.check_balance()}¥')
```



类的继承

- 通过继承 (Inheritance) 创建一个新类
 - 能自动拥有被继承类的所有功能
 - 可以添加属于自己的新功能
- 被继承的类称为基础类或父类(Base Class)
- 新类称为派生类或子类(Derived Class)

继承是OOP的一大特性，可极大减少重复代码



类的继承

- 语法很简单，在子类名的后面加上带括号的父类名即可

```
class SubClassName(BaseClassName):
```

- 在继承过程中，子类可以修改父类已有的方法
- 这种行为在OOP中称为方法重写(Method Overriding)
- 方法重写可以实现多态(Polymorphism)效果
 - 不同类的对象调用同一个方法时，会有不同的表现形态

类的继承

- 传递性
 - Person → Employee → Manager
- 单向性
 - 子类中新加的功能不会反向赋予父类

Manger类有多少个属性?

```
1 class Employee(Person): # 继承
2     def __init__(self, name, birth_date, role='普通员工'):
3         year, month, day = list(map(int, birth_date.split('-')))
4         super().__init__(year, month, day)
5         self.name = name
6         self.role = role
7
8     def get_info(self):
9         return f"姓名: {self.name}, 年龄: {self.get_age()}, 岗位: {self.role}"
10
11 class Manager(Employee): # 继承
12     def __init__(self, name, birth_date, department):
13         super().__init__(name, birth_date, '经理')
14         self.department = department
15
16     # 方法重写 (Override)
17     def get_info(self):
18         base_info = super().get_info()
19         return f"{base_info}, 管理部门: {self.department}"
20
21 # 创建对象列表
22 staff = [
23     Employee("小明", "2000-10-1"),
24     Employee("小满", "1998-3-15"),
25     Manager("老王", "1985-8-1", "技术部")
26 ]
27 # 循环展示多态性
28 for person in staff:
29     print(person.get_info())
```

姓名: 小明, 年龄: 25, 岗位: 普通员工

姓名: 小满, 年龄: 28, 岗位: 普通员工

姓名: 老王, 年龄: 41, 岗位: 经理, 管理部门: 技术部



课堂练习-2-3-2

- 通过继承 Wallet 实现一个儿童版微信钱包类 KidWallet
- 新增一个安全控制：当发红包金额大于10元时，直接拒绝，并提示“【警告】红包金额不能超过10元，已拦截!”



模块

- 若要调用上一讲的`calc_linear_gpa`，需要把代码复制-粘贴到当前Notebook中
- 真正的代码复用应是一次编写，无限复用
- 为此，需要将`calc_linear_gpa`放入一个模块中

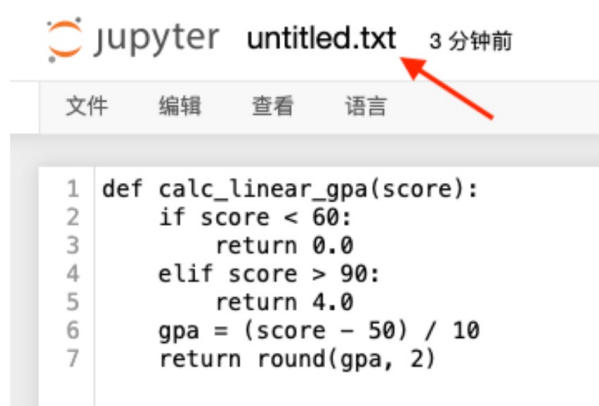


模块

- 模块(Module)是包含Python对象定义的.py文件
 - 对象可以是变量、函数、类等
- 模块名应该符合变量命名规则，同时体现模块功能
 - 命名使用全小写字母，若名字太长，则用下划线分隔以提高可读性
 - 例如tools.py、utils.py、math_tools.py、data_processor.py

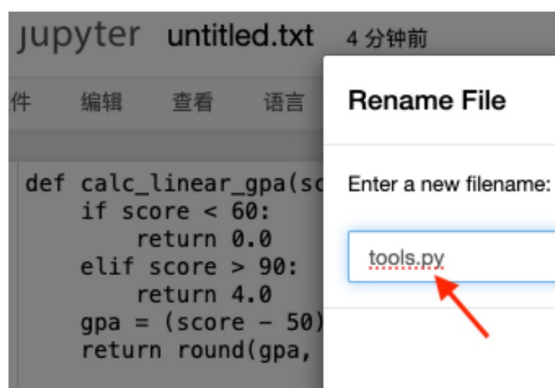
自定义模块

- 通过三步创建一个名为tools的模块



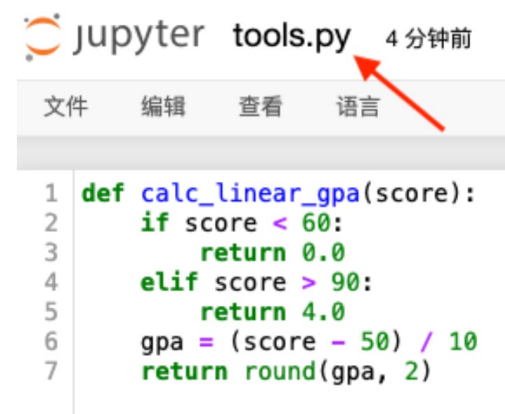
```
1 def calc_linear_gpa(score):
2     if score < 60:
3         return 0.0
4     elif score > 90:
5         return 4.0
6     gpa = (score - 50) / 10
7     return round(gpa, 2)
```

(a) 待重命名的.txt 文本文件



```
def calc_linear_gpa(score):
    if score < 60:
        return 0.0
    elif score > 90:
        return 4.0
    gpa = (score - 50) / 10
    return round(gpa, 2)
```

(b) 在文件重命名小窗中输入“tools.py”



```
1 def calc_linear_gpa(score):
2     if score < 60:
3         return 0.0
4     elif score > 90:
5         return 4.0
6     gpa = (score - 50) / 10
7     return round(gpa, 2)
```

(c) 完成重命名的.py 模块文件

图 2.3: 创建一个名为 tools 的自定义模块



自定义模块

- 使用保留字`import`导入自定义模块

```
import tools
```

【注意】对于特定模块，同一个进程只会执行一次导入。执行了一次`import tools`之后，对`tools.py`的后续修改不会同步更新

- 强制重载

```
import tools
import importlib
importlib.reload(tools)
```



定向导入与别名

- 可根据需要选择性导入特定功能
 - 语法: `from module_name import func_name`
- 为模块设置别名(Alias), 简化代码
 - 语法: `import module_name as alias`

```
from tools import calc_linear_gpa # 定向导入
from tools import calc_linear_gpa as gpa # 定向导入并为函数设置别名
# 调用模块中的函数
tools.calc_linear_gpa(89), gpa(89) # 输出: (3.9, 3.9)
```

常用第三方库的默认别名

```
import numpy as np
import pandas as pd
import streamlit as st
import matplotlib.pyplot as plt
```



常用内置模块

- 涉及系统与文件处理、数学与数据统计、时间与日期、数据格式等

返回本机的操作系统名称

```
import sys
sys.platform
```

表 2.10: 常用的 Python 内置模块

模块	功能
系统与文件类:	
os	操作系统接口 (第3.1节)
sys	系统参数 (命令行参数、解释器路径、模块搜索路径)
数学与数据类:	
math	数学运算 (三角函数、对数、常量 π 等)
random	生成随机数、随机选择
statistics	基础数学统计 (平均数、中位数等)
collections	高性能容器数据类型 (计数器、默认值字典、双端队列等)
时间与日期类:	
datetime	处理日期和时间 (获取当前时间、格式化输出)
time	时间戳转换、程序睡眠 (暂停)
数据格式类:	
csv	CSV 格式数据读写 (第3.2.2.1节)
json	JSON 格式数据读写 (第3.2.2.2节)
re	正则表达式匹配 (第4.1.2节)

常用内置模块

- 可在Jupyter Notebook中使用help函数查询特定模块的调用接口

```
1 import statistics as stat
2 help(stat)
```

```
=====
Function      Description
=====
mean          Arithmetic mean (average) of data.
fmean         Fast, floating-point arithmetic mean.
geometric_mean Geometric mean of data.
harmonic_mean Harmonic mean of data.
median        Median (middle value) of data.
median_low    Low median of data.
median_high   High median of data.
median_grouped Median, or 50th percentile, of grouped data.
mode          Mode (most common value) of data.
multimode     List of modes (most common values of data).
quantiles     Divide data into intervals with equal probability.
=====
```

Calculate the arithmetic mean ("the average") of data:

```
1 stat.mean([1, 2, 3, 4, 100]), stat.median([1, 2, 3, 4, 100])
```

(22, 3)

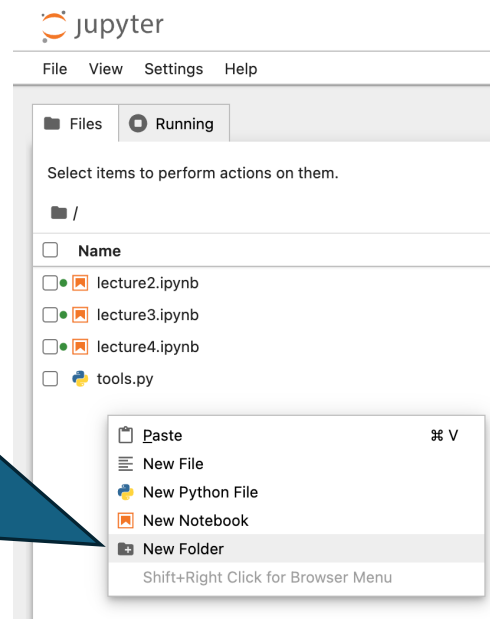
库

- 库(Package)是包含一个或多个模块的目录

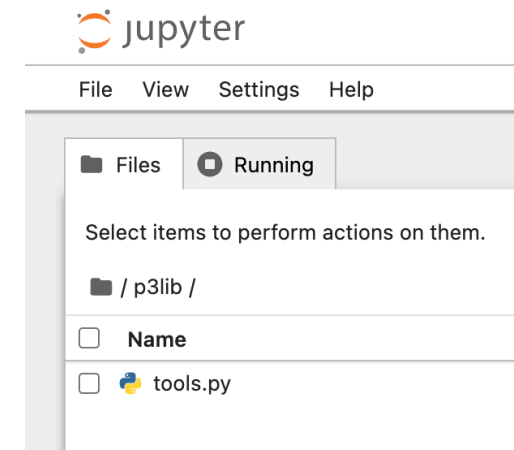
在当前工作路径下新建一个名为p3lib的自定义库

在Notebook主页空白处，点击鼠标右键，在小窗列表中选中“新建文件夹”，并重命名为p3lib

第1步. 创建名为p3lib的文件夹



第2步. 将tools.py拖拽到p3lib中





库的使用

- 与导入模块的语法一样，我们也使用`import`导入库

示例代码 2.41 导入一个名为 `p3lib` 的自定义库并使用其模块

```
1 import p3lib
2 from p3lib import tools
3 from p3lib.tools import calc_linear_gpa
4
5 score = 90
6 print(p3lib.tools.calc_linear_gpa(score)) # 输出 4.0
7 print(tools.calc_linear_gpa(score)) # 输出 4.0
8 print(calc_linear_gpa(score)) # 输出 4.0
```

注意观察这几种
导入方式的差别



第三方库

• Python的开源生态提供了大量免费、高效的第三方库

表 2.11: 本书教学案例中用到的 Python 第三方库。在后续章节中，我们一般采用 `from xx import yy` 的方式，从库 `xx` 中定向导入所需模块 `yy`，以提升代码可读性和执行效率。

库名	主要用途	导入语句	对应章节
AKShare	财经数据接口库	<code>import akshare</code>	8.3.2
beautifulsoup4	HTML 文档解析	<code>import bs4</code>	3.2.2.3
DrissionPage	网页自动化工具	<code>import DrissionPage</code>	3.3.4
huggingface_hub	Hugging Face 客户端库	<code>import huggingface_hub</code>	5.4.1
jieba	中文分词	<code>import jieba</code>	4.2.2, 5.2.2.3
joblib	Python 对象序列化与保存	<code>import joblib</code>	5.2.1.4
matplotlib	绘制折线图、直方图、散点图等图表	<code>import matplotlib</code>	6.1
NumPy	数值计算	<code>import numpy as np</code>	4.3
OpenAI	大模型 API 调用	<code>import openai</code>	8.1.1
openpyxl	Excel 表格处理	<code>import openpyxl</code>	3.2.3.2
Pandas	表格数据处理与分析	<code>import pandas as pd</code>	4.1.1, 4.4
PIL	图像数据读写	<code>import PIL</code>	3.2.3.1
pytest	代码测试	<code>import pytest</code>	7.3.1
python-docx	Word 文档处理	<code>import docx</code>	3.2.3.3
PyTorch	深度学习	<code>import torch</code>	5.3
pyvis	绘制关系图	<code>import pyvis</code>	6.2
requests	HTTP 请求与响应处理	<code>import requests</code>	3.3.1.3
Scikit-learn	机器学习	<code>import sklearn</code>	5.2
sentence_transformers	基座模型	<code>import sentence_transformers as sentrans</code>	5.4
streamlit	Web 应用开发	<code>import streamlit as st</code>	6.5, 7.2
tqdm	屏幕输出进度条	<code>import tqdm</code>	5.3.5.3
transformers	Transformer 系列模型库	<code>import transformers</code>	5.3.5
wordcloud	绘制词云	<code>import wordcloud</code>	6.3



第三方库

- 为便于安装，先更换下载源以避免网络问题
 - 在Anaconda 命令行环境下，执行下列命令

添加 conda 镜像通道

```
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/free/  
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main/
```

在安装包时显示下载URL

```
conda config --set show_channel_urls yes
```

修改 pip 的默认镜像源

```
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```



第三方库

- 示例：安装jieba

在Anaconda命令行界面中输入下列命令

```
pip install jieba
```

```
Anaconda Prompt
(base) C:\Users\24246>pip install jieba
Defaulting to user installation because normal site-packages is not writeable
Collecting jieba
  Downloading jieba-0.42.1.tar.gz (19.2 MB)
    19.2/19.2 MB 9.6 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: jieba
  Building wheel for jieba (setup.py) ... done
  Created wheel for jieba: filename=jieba-0.42.1-py3-none-any.whl size=19314474 sha256=22882f69f0b60c9041a31fb74e9f06440b9ff5482011c681811d6910d6e7b58b
  Stored in directory: c:\users\24246\appdata\local\pip\cache\wheels\08\al\3\5c8ac52cc2f5782ffffc34c95c57c8e5ecb3063dc69541ee7c
Successfully built jieba
Installing collected packages: jieba
Successfully installed jieba-0.42.1
(base) C:\Users\24246>
```

安装完成后，再在命令行中输入命令 `python -c "import jieba"`
如无报错信息，则表明jieba已安装到位



新手常见错误

- 只要动手编程，就可能产生代码错误
- 错误主要有两种
 - 语法错误(Syntax Error): 可由Python 解释器发现并报告
 - 逻辑错误(Logic Error): 存在逻辑错误的代码可以“正常”运行不会报错，但输出结果往往与预期不符

语法错误

- 中英文标点符号混淆

```
print("你好")
```

```
Cell In[1], line 1
```

```
print("你好")
```

^

```
SyntaxError: invalid character ' ) ' (U+FF09)
```

(a) 函数调用时用了中文右括号

```
if score >= 60:  
    print('pass')
```

```
Cell In[3], line 1
```

```
if score >= 60:
```

^

```
SyntaxError: invalid character ' : ' (U+FF1A)
```

(b) if 语句中用了中文冒号

```
list = [1, 2, 3]
```

```
Cell In[2], line 1
```

```
list = [1, 2, 3]
```

^

```
SyntaxError: invalid character ' , ' (U+FF0C)
```

(c) 列表中以中文逗号作为分隔符

```
message = "你好"
```

```
Cell In[4], line 1
```

```
message = "你好"
```

^

```
SyntaxError: unterminated string literal (det
```

(d) 字符串以中文双引号结束

图 2.5: 因使用中文标点导致的语法错误



语法错误

- 未注意区分变量名大小写

jupyter common_error Last Checkpoint: 1 minute ago

File Edit View Run Kernel Settings Help Trusted

Code

JupyterLab Python 3 (ipykernel)

变量名大小写不敏感

```
[1]: score = 90
     print(Score)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[1], line 2
      1 score = 90
----> 2 print(Score)

NameError: name 'Score' is not defined
```

语法错误

- 缩进错误

```
1 import math
2
3 def calc_circle_area(radius):
4 s = radius**2
5 res = math.pi * s
6 return res
7
8 calc_circle_area(1)
```

```
Cell In[1], line 4
s = radius**2
^
IndentationError: expected an in
```

(a) 忘记缩进

```
1 import math
2
3 def calc_circle_area(radius):
4     s = radius**2
5     res = math.pi * s
6     return res
7
8 calc_circle_area(1)
```

```
File <string>:5
res = math.pi * s
^
IndentationError: unindent does
```

(b) 缩进位数不一致

```
1 import math
2
3 def calc_circle_area(radius):
4     s = radius**2
5     res = math.pi * s
6     return res
7
8 calc_circle_area(1)
```

```
Cell In[1], line 5
res = math.pi * s
^
IndentationError: unexpected ind
```

(c) 意外缩进

图 2.6: 缩进错误的几种情况



语法错误

- 赋值与判等混淆

```
1 if a = 10:  
2     print('a等于10')
```

```
Cell In[1], line 1  
    if a = 10:  
        ^
```

SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?

图 2.7: 将赋值当成判等来使用



逻辑错误

- 浮点数精度陷阱

示例代码 2.43 浮点数精度陷阱。预期输出"Equal"，实际输出"Not Equal"

```
1 a = 0.1 + 0.2
2 if a == 0.3:
3     print("Equal")
4 else:
5     print("Not Equal") # 输出 Not Equal
```

正确做法是改用 `math.isclose(a,0.3)`

逻辑错误

- 在遍历列表时修改列表

示例代码 2.44 在遍历列表时修改列表

```
1 nums = [1, 2, 2, 3, 4]
2 for i,n in enumerate(nums):
3     print(i, n)
4     if n == 2:
5         nums.remove(n)
6 print(nums) # 期望 [1, 3, 4], 结果 [1, 2, 3, 4]
7 # 正确做法: 列表推导式
8 nums = [n for n in nums if n!=2]
```



逻辑错误

- 浅复制隐患
 - 容器类对象(列表、集合、词典等)的默认复制为浅复制(Shallow Copy)
 - 浅复制可能导致误操作
 - 使用深复制(Deep Copy)避免潜在的误操作

示例代码 2.45 浅复制与深复制

```
1 a = [[1,0], [0,1]]
2 b = list(a) # b是a的浅复制
3 b[0][0] = 2
4 print(a) # 输出: [[2, 0], [0, 1]]。a被修改了!
5 # 正确做法
6 import copy
7 b = copy.deepcopy(a) # 对b的任何修改都不会影响a
```



逻辑错误

- 变量作用域(Variable Scope)错误
- 变量作用域是指一个特定变量可被访问的范围
- 对于一个函数而言，其函数体涉及的变量的作用域仅限于该函数体
 - 因此这些变量也被称为局部变量(Local Variable)
- 在函数外部定义的变量被称为全局变量(Global Variable)

逻辑错误

- 根据Python访问变量的规则，当查找一个变量的时候，会先从局部作用域开始查找，若未找到，则再在全局作用域中搜索

示例代码 2.46 变量作用域

```
1 count = 0 # 全局变量
2
3 def func1():
4     print(count) # 不修改全局变量可以访问全局变量
5
6 def func2():
7     count = count + 1 # 修改同名全局变量，则Python会认为它是一个局部变量，报错：UnboundLocalError
8
9 func1()
10 func2()
11 # 推荐做法
12 def func3(c):
13     return c+1
14 count = func3(count)
```

在函数fun2内部，count会被当作一个与全局变量同名的局部变量

逻辑错误

- 值相等与对象标识相等混用

示例代码 2.47 对比值相等 (==) 与对象标识相等 (is)

```
1 a = [1, 2, 3]
2 b = [1, 2, 3]
3 print(a == b) # True (内容一样)
4 print(a is b) # False (内存地址不同)
5 # 因小整数缓存机制导致的例外情形
6 x = 101
7 y = 101
8 print(x is y) # True
```

最后一行，逻辑上x和y应指向不同对象。为什么输出True?